

TimeSage-EV: A Live Benchmark for Agentic Time Series Analysis in Evolving Environments

Qingren Yao^{1*}, Yaxuan Kong^{2,3*}, Yuqi Nie, Yichen Li³, Stefan Zohren², Anna Vettoruzzo¹,
Qingsong Wen⁴, Ming Jin^{5†}, Joaquin Vanschoren^{1†}

¹Eindhoven University of Technology, ²University of Oxford, ³VulpiVox Intelligence,

⁴Squirrel Ai Learning, ⁵Griffith University

q.yao@tue.nl; yaxuan.kong@eng.ox.ac.uk; mingjinedu@gmail.com; j.vanschoren@tue.nl



Dataset



Code

Abstract

Time series analysis in high-stakes domains relies on recurring data releases, where new observations can alter the evidence base and the validity of later conclusions. Existing time series QA benchmarks mostly rely on fixed snapshots, leaving temporal validity and cutoff-aware evidence use unevaluated. We introduce **TimeSage-EV**, a live benchmark for agentic time series analysis in evolving environments. It tracks 60 real institutional scenarios across 6 domains, comprising 1,485 scenario-period QA pairs from Feb 2023 to May 2026 and spanning monthly, weekly, daily, and irregular release cadences. At each period, large language model (LLM) agents receive time series data and source reports, while the withheld target release provides ground truth. TimeSage-EV evaluates state identification, data summarization, and outlook reasoning. Experiments with frontier LLM agents and **TimeSage-1.0**, a novel self-evolving agent with a reusable analytical skill library, reveal significant performance gaps across model tiers and recurring failures in temporal validity, exogenous context use, and adaptation. We release TimeSage-EV as a research resource with monthly updates, code, a leaderboard, and failure-mode analyses.

Models (LLMs) to maintain temporal validity while reasoning over dynamic multimodal evidence.

Existing benchmarks only partially cover this setting, with two key limitations. *(1)* Time series question answering (QA) benchmarks, such as ChatTime-TSQA (Wang et al., 2025), TimeMQA (Kong et al., 2025), and TSRBench (Yu et al., 2026), test temporal reasoning over time series, but usually construct fixed QA pairs over predefined time series tasks rather than evolving workflows. *(2)* Agent-centric benchmarks evaluate coding (Jimenez et al., 2024), web navigation (Zhou et al., 2024; Mialon et al., 2024), multi-hop retrieval (Wei et al., 2025), and tool use (Yao et al., 2025), but are not designed around recurring time series analysis. They rarely require agents to track time-indexed evidence, compare new observations with historical context, and synthesize time series with institutional reports. Thus, it remains unclear whether LLM agents can perform such analytical work as the evidence changes over time.

Constructing a benchmark for this dynamic setting presents three challenges. *(1)* Each scenario must be drawn from a stable institutional source with a recurring release schedule, ensuring the evaluation reflects real-time monitoring rather than synthetic snapshots. *(2)* Each target period must impose a strict information cutoff. Agents can only access cumulative data and source documents published prior to the target release, while the target release itself is withheld for ground truth construction and evidence verification. *(3)* Evaluation cannot be restricted to closed-form choices. To align with practical applications, it must assess both key analytical tasks and human-readable reports. These requirements motivate a benchmark design with timestamp-aware ground truth, traceable evidence, cutoff-valid inputs, and metrics that explicitly separate factual correctness from report faithfulness.

To address these challenges, we introduce **TimeSage-EV**, a live benchmark for **agentic time**

1 Introduction

Across high-stakes domains, from finance and public health to transportation, time series analysis is not a static prediction task but a recurring workflow. Analysts must continuously ingest new data releases, update historical context, and synthesize numeric trends with institutional reports. Crucially, each new release shifts the available evidence distribution, requiring strict adherence to timestamp-aware information cutoffs. We formalize this recurring workflow as agentic time series analysis in evolving environments, requiring Large Language

*Equal contribution. †Corresponding author.

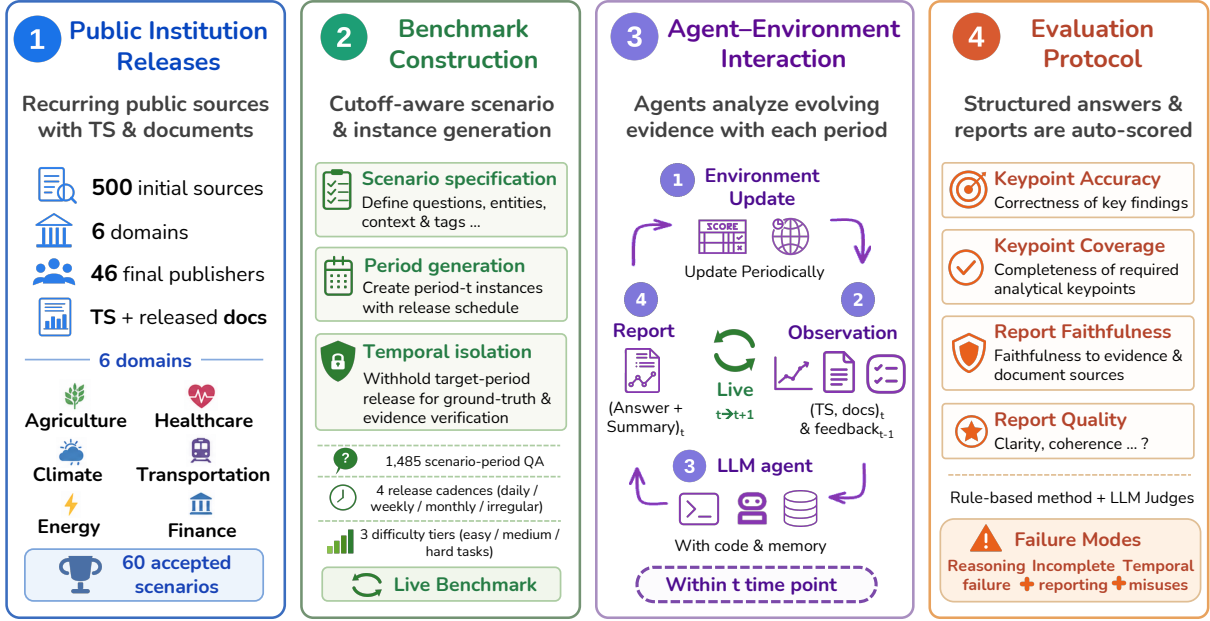


Figure 1: Overview of TimeSage-EV, supporting agentic time series tasks and a live leaderboard. TS: time series.

series analysis in evolving environments. As shown in Figure 1, each scenario is a recurring task anchored to an institutional source, defined by a fixed domain, release cadence, difficulty tier and set of questions. TimeSage-EV tracks 60 scenarios across six domains from February 2023 to May 2026, covering agriculture, climate, energy, finance, healthcare, and transportation. The scenarios are organized into three difficulty tiers: easy tasks focus on state identification; medium tasks require change detection, composition analysis, and event detection; and hard tasks require attribution and forecast. Each scenario unfolds over monthly, weekly, daily, or irregular release periods.

For each target period, the benchmark instantiates specific questions and structured answer fields, comprising 1,485 scenario-period QA pairs. The agent receives pre-cutoff time series data and documents, while the target-period release is withheld for verification. The question types are annotated with a 27-tag taxonomy grouped into six families (Appendix A.3.1 Table 6): state identification, change detection, composition analysis, event detection, attribution, and outlook. The agent answers the questions and compiles them into a report. Critically, as new observations arrive in periods, the agent must update its judgments and revise its report adhering to the rolling evidence cutoff.

Our evaluation protocol scores the output along four axes: keypoint accuracy, keypoint coverage, report faithfulness, and report quality. This distin-

guishes agents that compute correct answers from those that generate fluent but weakly grounded reports. We then evaluate frontier LLM agents and **TimeSage-1.0**, a self-evolving agent with a reusable skill library, on **TimeSage-EV**. The results reveal large performance gaps across difficulty tiers and failures in memory management, temporal misuse, and report synthesis. These findings show that current agents struggle to maintain cutoff-correct evidence and to adapt to new releases, offering diagnostic guidance for reliable agents.

Our contributions are summarized as follows.

- We formalize a novel task for **agentic time series analysis in evolving environments**, requiring temporal isolation, cutoff-valid evidence use, and recurring analytical updates.
- We construct **TimeSage-EV**, a live benchmark¹ comprising 60 scenarios across 6 domains, 3 tiers, and 1,485 scenario-period questions with ground truth and traceable evidence.
- We design a rigorous **multi-axis evaluation protocol** measuring keypoint accuracy, keypoint coverage, report faithfulness, and report quality across structured answers and analytical reports.
- We benchmark frontier LLM agents and the diagnostic **TimeSage-1.0**, uncovering failure modes in memory management, temporal misuse, and report synthesis to guide future research.

¹The benchmark is kept live through a maintenance script that runs automatic updates monthly. See Appendix A.2.

2 Related Work

Prior work spans language-grounded time series analysis and long-horizon live agent benchmarks. However, live benchmarks for agentic time series analysis in evolving environments remain underexplored, particularly in scenarios where agents must repeatedly analyze recurring data releases while adhering to cutoff-valid numerical and documentary evidence. Table 1 summarizes representative benchmarks across these dimensions.

Table 1: Comparison of representative benchmarks. ✓ denotes support, △ partial support, and ✗ not covered.

Benchmark	TS	Text	Agent	Live	Evol.	Cutoff
Time-LLM (Jin et al., 2024)	✓	✗	✗	✗	✗	✗
ChatTime (Wang et al., 2025)	✓	✓	✗	✗	✗	✗
Time-MQA (Kong et al., 2025)	✓	✓	✗	✗	✗	✗
TimeSeriesExam (Cai et al., 2024)	✓	✓	✗	✗	✗	✗
WebArena (Zhou et al., 2024)	✗	✓	✓	✗	✗	✗
LiveBench (White et al., 2025)	✗	△	△	✓	✗	✗
ForecastBench (Karger et al., 2025)	✗	✗	△	✓	✗	✗
TimeSeriesGym (Cai et al., 2025)	✓	✓	✓	✗	✗	✗
TimeSage-MT (Kong et al., 2026)	✓	✓	✓	✗	✗	✗
TimeSage-EV	✓	✓	✓	✓	✓	✓

TS: numerical time series evidence; Text: textual or documentary evidence; Evol.: the same scenario is revisited across release periods; Cutoff: period-specific evidence isolation.

Language-grounded time series analysis. Time series analysis is increasingly framed as a language-facing task that connects numerical patterns with instructions and domain context. LLMTime (Gruver et al., 2023) and Time-LLM (Jin et al., 2024) recast time series for language-model use, while ChatTime (Wang et al., 2025), ChatTS (Xie et al., 2025), Time-MQA (Kong et al., 2025), TSRBench (Yu et al., 2026), TimeSeriesExam (Cai et al., 2024), TimeSage-MT (Kong et al., 2026), and TemporalBench (Weng et al., 2026) evaluate temporal reasoning through natural-language questions. Beyond question answering, TimeSeriesGym (Cai et al., 2025) evaluates LLMs on end-to-end time series machine learning tasks. These settings remain snapshot-based, with fixed questions over static evidence, and do not evaluate how agents update analyses across recurring releases.

Long-horizon live agent benchmarks. LLM agents are evaluated through tool use, retrieval, code execution, and memory, as in ReAct (Yao et al., 2023), Toolformer (Schick et al., 2023), and AutoGen (Wu et al., 2024). Benchmark tasks span web navigation, dialogue, and software engineering, including WebArena (Zhou et al., 2024), GAIA (Mialon et al., 2024), Tau-Bench (Yao et al., 2025), and SWE-bench (Jimenez et al., 2024). Live benchmarks such as LiveBench (White et al., 2025)

and LiveCodeBench (Jain et al., 2024) reduce contamination through refreshed task pools, while ForecastBench (Karger et al., 2025) evaluates predictions after outcomes resolve. Yet, these settings remain refreshed snapshots, rather than evolving scenarios with recurring releases and cumulative numerical-documentary evidence.

Positioning TimeSage-EV. Existing work leaves a critical gap at the intersection of these two lines: live, continuous agent evaluation for recurring time series analysis. **TimeSage-EV** addresses this gap by evaluating agentic time series analysis in evolving environments, where agents revisit institutional releases, combine evolving numerical and textual evidence, respect time cutoffs, and produce both structured answers and grounded reports. TimeSage-EV moves beyond static QA task to evaluate evolving time series analysis as a dynamic, long-horizon agentic workflow (Appendix A.1).

3 Methodology

3.1 Task Formulation

We formulate **agentic time series analysis in evolving environments** as a recurring task over sequential release periods. In **agentic time series analysis**, an agent must utilize tools to inspect time series, consult cutoff-valid documents, answer structured questions, and generate an evidence-based report. An **evolving environment** presents this task under changing information states, as new evidence arrives across subsequent periods (Figure 2). We formalize this setting below and provide expanded definitions in Appendix A.2.

Scenario Task. An **environment** for scenario s defines a recurring analytical task over time:

$$\mathcal{E}_s = (D_s, Q_s, A_s, \{e_{t_k}^{(s)}\}_{t_k \in \mathcal{T}_s}),$$

where D_s denotes the domain background, Q_s is the recurring question template, A_s is the structured answer schema, and $\{e_{t_k}^{(s)}\}$ is the sequence of evidence states across release periods $\mathcal{T}_s = \{t_1, \dots, t_T\}$. The task specification (D_s, Q_s, A_s) remains fixed within a scenario, while $e_{t_k}^{(s)}$ changes as new releases become available.

Period Instance. For each target period t_k , the period instance is defined by the environment state

$$e_{t_k}^{(s)} = (q_{t_k}, X_{\leq t_k}, R_{< t_k}, W_{\leq t_k}^{(s)}),$$

where q_{t_k} is the natural-language question, $X_{\leq t_k}$ is the cumulative time series table up through the

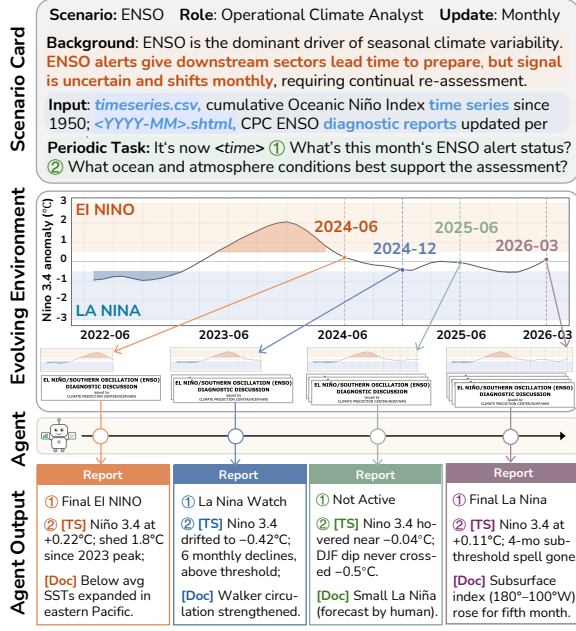


Figure 2: Evolving environment in TimeSage-EV with ENSO scenario. TS: time series, Doc: source document.

target period, and $R_{<t_k}$ contains source documents released strictly before the target period. The target period release R_{t_k} is withheld from the agent and used only for ground truth construction and evidence verification. Thus, the time series is visible, but the document is not. Finally, $W_{\leq t_k}^{(s)}$ denotes external web evidence available prior to the cutoff. By default, $W_{\leq t_k}^{(s)} = \emptyset$, and it is enabled only for scenarios with cutoff-valid retrieval.

Agent State and Output. Within this environment, the LLM agent maintains a state S_{t_k} , which may encompass memory, reasoning traces, or reusable analysis artifacts. Given a period instance, the agent returns

$$(\hat{y}_{t_k}, \hat{r}_{t_k}, S_{t_{k+1}}) = \text{Agent}_s(e_{t_k}^{(s)}, S_{t_k}),$$

where $\hat{y}_{t_k}$ is a schema-conformant structured answer and $\hat{r}_{t_k}$ is a cutoff-grounded report. Stateless agents use $S_{t_k} = \emptyset$. Figure 2 illustrates this **evolving** process. As the benchmark progresses from t_k to t_{k+1} , the task specification remains fixed, but the evidence state expands:

$$X_{\leq t_k} \subseteq X_{\leq t_{k+1}}, \quad R_{<t_k} \cup \{R_{t_k}\} \subseteq R_{<t_{k+1}}.$$

Thus, each step introduces a new evidence state in which the time series is extended, the previous target release becomes prior evidence, and the next target release is withheld for evaluation. The agent must continuously revise its judgments without violating the time cutoff. As future releases are published, the same process creates new temporally

isolated evaluation periods, making TimeSage-EV a continuously updated **live benchmark**.

3.2 Benchmark Construction

TimeSage-EV is built from public institutional releases that recur on a fixed cadence, and every evaluation period of every scenario is reproducible from those releases. The pipeline proceeds in three stages: source-side data collection, scenario-side generation of period-level instances, and a quality control pass that enforces temporal isolation. Figure 3 reports corpus statistics, while Figure 4 sketches the pipeline. Full prompts, scenario checklists, and audit attestations are provided in Appendix A.3.

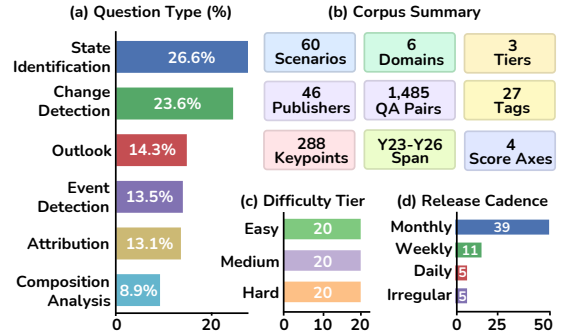


Figure 3: TimeSage-EV corpus statistics.

3.2.1 Data Collection

Step I: Domain scanning and source registry. We built a registry of 500 public institutional sources across six domains: agriculture, climate, energy, finance, healthcare, and transportation. Candidate sources include APIs, periodic briefs, and statistical reports from government agencies and international organizations. For each source, we record metadata, including publisher, source type, release cadence, historical span, and data format.

Step II: Attribute-based filtering. We retain sources that are public, recurring, historically deep enough for repeated instantiation, grounded in timestamped time series, and tied to answers traceable to artifacts. We exclude sources requiring private access and lacking period alignment. We balance domain coverage, source type, and release cadence so that the corpus spans monthly, weekly, daily, and irregular schedules.

Step III: Source validation. Each surviving source is validated for all target periods before promotion. We verify that source artifacts can be recovered, time series remain stable and ordered, cumulative histories can be assembled for each tar-

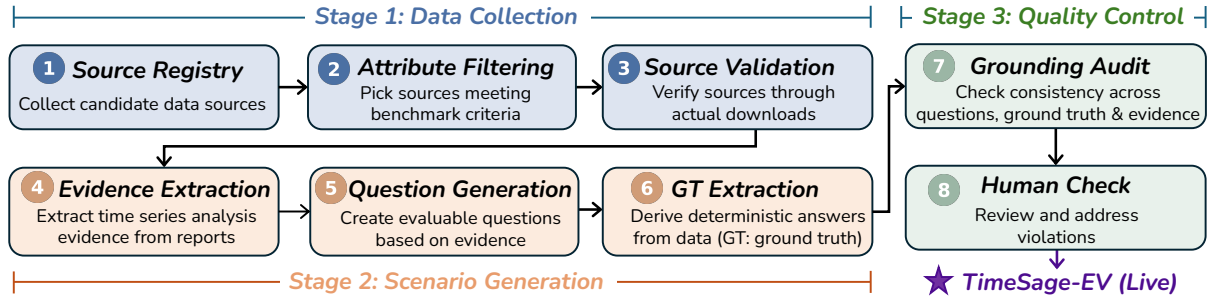


Figure 4: Overview of benchmark generation pipeline. Benchmark data is updated monthly via automated scripts.

get period, and ground-truth fields are traceable to time series observations or quoted document spans.

3.2.2 Scenario Generation

Step IV: Evidence collection. For each accepted sources, we construct period-level evidence packages. Each package represents the agent-visible state at a given period, containing the time series and documents accumulated up to that point. We extract background information from the source description and time series-relevant analytical text from the documents. These materials provide the basis for subsequent question generation.

Step V: Question generation. Using the evidence in Step IV, we generate questions and export a scenario-level contract for each source. To do this, we adapt question, answer-schema, and scoring templates from a 27-tag taxonomy, spanning six question categories (Appendix A.3.1 Table 6), into a scenario-specific contract. LLMs are used for this adaptation, while the predefined question-answer-scoring structure is preserved. The contract can be instantiated in every period. The question distribution varies by difficulty: easy scenarios focus on state identification; medium scenarios require temporal pattern identification; and hard scenarios involve analytical reasoning that combines numerical changes with document-based explanations. This difficulty cliff is shown in Appendix A.7 Table 12: keypoint answer accuracy declines monotonically from easy to hard across all evaluated LLMs.

Step VI: Ground-truth extraction. For each instance, we derive reproducible ground truth according to the scenario contract. For each period, we follow the answer schema and scoring method specified in the scenario contract to construct ground truth, either through computation or LLM-assisted evidence finding, depending on the field type. For fields defined over the time series, we apply deterministic rules to the time series table, including threshold-based classification, multi-variable com-

parison, and cross-period comparison. For fields requiring textual evidence, we use the withheld target-period source during answer construction and verification, transcribing the statements or explanations from the document.

3.2.3 Quality Control

Every scenario undergoes an automated grounding audit and human review before release. The grounding audit checks schema validity, temporal consistency, variable and answer-space validity, deterministic answer derivability, evidence support, leakage prevention, and instance-level completeness. Three human reviewers then verify the ground truth, the clarity of the questions, and the grounding of the evidence. A scenario is released only after passing both stages. The final corpus contains 60 scenarios from 6 domains and 1,485 period-level instances spanning February 2023 to May 2026, with a balanced 20/20/20 difficulty-tier split, as summarized in Figure 3.

3.3 Evaluation Protocol

Each agent run produces two outputs at each period: a structured answer and a free-form report. We score them on four axes: (1) **keypoint accuracy**, which checks whether the structured answer matches the ground truth; (2) **keypoint coverage**, which checks whether the report mentions the keypoints; (3) **report faithfulness**, which checks whether the report’s claims are supported by cutoff-valid evidence rather than future or unsupported information; and (4) **report quality**, which checks whether the report is clear, organized, and appropriate for the scenario. The protocol combines rule-based scoring for fields with fixed answer formats, such as labels, yes/no values, and numbers, with LLM-based judging for the report’s coverage, faithfulness, and writing quality. Each axis is reported on a 0–100 scale, and the run-level score is the average of the scored axes. We report both inde-

pendent evaluation, where the agent starts fresh for each period, and sequential evaluation, where the agent can carry state within a scenario but receives only prior-period evidence. Figure 5 shows a scenario example with the evaluation protocol. More details on the judge are shown in Appendix A.4.

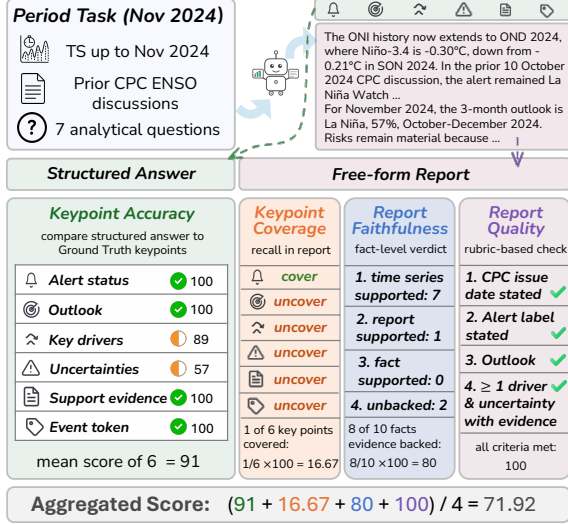


Figure 5: Evaluation for the El Niño–Southern Oscillation (ENSO) monitoring scenario. TS: time series.

4 Experiments

4.1 Environment Setting

Each evolving scenario is instantiated as a sandbox that periodically releases time series analysis tasks and coordinates information exposure, evaluation, and feedback. In each period, the environment publishes the task and exposes only causally admissible information prior to that time point. The agent completes the task within this temporally isolated sandbox and submits its output. Once the scenario source releases the corresponding report, the environment collects the ground-truth outcome, evaluates the output, returns feedback, and advances to the next period, repeating the cycle. With a collection cutoff in May 2026, we snapshot the latest 24 periods per scenario, while an automated pipeline continuously pulls from the scenario sources so each environment updates at its own release cadence, and the benchmark remains live over time. All experiments in this paper use these 24 periods.

4.2 Experimental Setting

Baseline Models. We use smolagent (Roucher et al., 2025) as the baseline, equipped with memory, search, and code execution. A memory-compression tool can be invoked by the agent

or triggered automatically when the context approaches the token limit, as detailed in Appendix A.6. We evaluate six LLMs under this harness: Sonnet-4.6 (Anthropic, 2026), GPT-5.4 (OpenAI, 2026), Kimi-K2.6 (Kimi Team, 2025), Qwen-3.5-397B-A17B-NVFP4-TBC (Qwen Team, 2026), Devstral-2-123B-Instruct-2512 (Mistral AI, 2025), and Gemma-4-31B-IT-NVFP4 (Google DeepMind, 2026).² All models use temperature=0 with a 32,768-token output budget; all other hyperparameters remain at provider defaults. Each rollout is limited to 20 steps or 2,700 seconds; runs that exhaust either budget without producing an answer are counted as failures. For the LLM judge axes (coverage, faithfulness, and quality), we use DeepSeek-V4-Flash (DeepSeek-AI, 2026), with the prompts shown in Appendix A.5.2 and verification in Appendix A.7.2.

TimeSage-1.0. To examine whether agents can self-improve from feedback over long-horizon tasks, we propose TimeSage-1.0, a smolagent-based harness augmented with a self-evolving skill-library management system (see Appendix A.6 for more details). The system supports skill induction, quality checking, and skill retrieval, allowing agents to convert reasoning traces into reusable workflows and deploy them in subsequent periods.

4.3 Main Results

Overall performance. Across the full benchmark, GPT-5.4 achieves the strongest score (86.0), followed by Qwen-3.5-397B (84.2) and Sonnet-4.6 (82.4), as summarized in Figure 6. Performance is near-saturated on easy tasks, where all models score above 92, but drops as the task requires richer analytical synthesis. On hard tasks, the best model reaches only 72.7, while Kimi-K2.6 and Gemma-4-31B fall to 59.3 and 53.1, respectively. This tier-wise gap indicates that agents can handle basic time series analysis tasks, including state and temporal pattern identification. However, *large-scale evidence extraction and external-information reasoning remain major bottlenecks in hard scenarios.*

Token cost. As shown on the right side of Figure 6, total token cost varies by more than $4\times$ across models, from 84M for GPT-5.4 to 364M for Sonnet-4.6. GPT-5.4 achieves the highest overall score while using the fewest tokens, showing a clear efficiency advantage. In contrast, Sonnet-4.6 and other

²Sonnet-4.6 and GPT-5.4 are accessed through the OpenRouter API (<https://openrouter.ai>), while the remaining LLMs are served via local vLLM deployments.

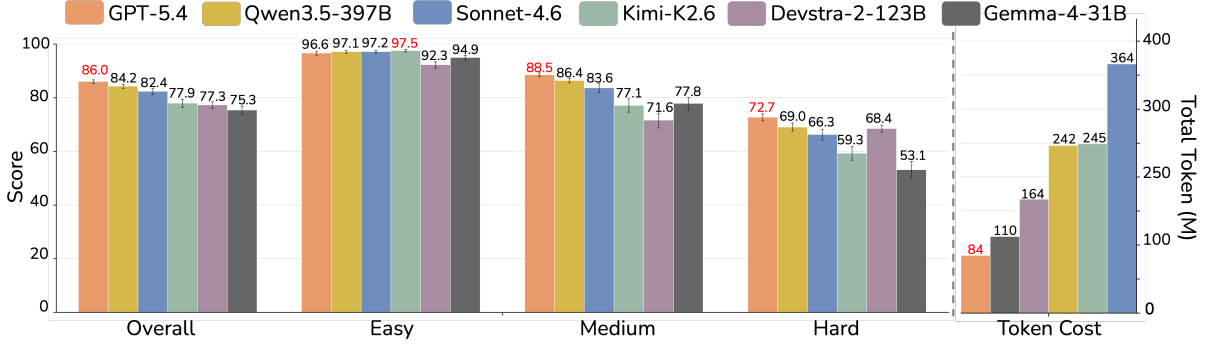


Figure 6: **Left:** Outcome scores across difficulty tiers and overall; error bars are per-difficulty-level percentile bootstrap 95% confidence intervals. **Right:** Total token cost.

Table 2: Per-question-type scores, reported as mean with bootstrapped 95% CI in the format x_{-low}^{+high} . Cell shading uses a shared scale (low high); the best result in each column is shown in bold.

Model	State ID	Change Det.	Comp. ID	Event Det.	Attribution	Forecast
🌟 Sonnet-4.6	90.9 ^{+1.2} _{-1.2}	92.9 ^{+1.5} _{-1.7}	85.6 ^{+2.7} _{-2.7}	67.9 ^{+2.2} _{-2.1}	42.6 ^{+3.0} _{-3.2}	41.2 ^{+4.1} _{-4.0}
🌀 GPT-5.4	93.8 ^{+0.8} _{-0.9}	96.0 ^{+1.2} _{-1.2}	86.9 ^{+2.4} _{-2.7}	75.4 ^{+1.9} _{-1.9}	46.2 ^{+3.2} _{-3.4}	42.7 ^{+4.1} _{-3.9}
🌀 Kimi-K2.6	84.2 ^{+1.8} _{-1.7}	84.9 ^{+2.4} _{-2.4}	80.8 ^{+3.2} _{-3.1}	61.0 ^{+2.6} _{-2.5}	39.3 ^{+3.3} _{-3.4}	33.4 ^{+3.9} _{-3.7}
🌀 Qwen-3.5-397B	92.6 ^{+0.9} _{-0.9}	97.0 ^{+0.9} _{-1.0}	86.8 ^{+2.4} _{-2.5}	74.1 ^{+1.9} _{-1.9}	46.1 ^{+3.3} _{-3.2}	43.0 ^{+4.0} _{-3.8}
🌀 Devstral-2-123B	83.4 ^{+1.5} _{-1.6}	76.5 ^{+2.8} _{-2.6}	79.7 ^{+3.1} _{-3.2}	64.7 ^{+2.4} _{-2.4}	46.7 ^{+3.2} _{-3.0}	42.8 ^{+4.1} _{-4.3}
🌀 Gemma-4-31B	82.7 ^{+1.7} _{-1.8}	80.2 ^{+2.4} _{-2.7}	77.3 ^{+3.4} _{-3.3}	55.7 ^{+2.7} _{-2.6}	37.3 ^{+3.4} _{-3.2}	34.8 ^{+4.0} _{-3.8}

high-cost models fail to lead in accuracy. These results suggest that *reasoning efficiency drives performance in long-horizon time series analysis*.

Live leaderboard. Figure 7 traces how overall scores evolve under continuous testing, where each LLM’s in-context history accumulates from period $t - 23$ to the cutoff time. After an initial three-period ramp-up, all models reach a high plateau, indicating that early context is useful. The trajectories then diverge: GPT-5.4 maintains the highest and most stable score with no degradation, showing strong long-horizon memory robustness, whereas Gemma-4-31B and Kimi-K2.6 decline steadily, the former falling from 80 to nearly 63. This late-stage drop suggests that *long-horizon history retrieval becomes a bottleneck for weaker LLMs in time series analysis*. This is a key advantage of TimeSage-EV over static benchmarks: rather than evaluating isolated snapshots, it exposes whether LLMs remain reliable under long-term deployment.

Per-question-type analysis. The per-question-type results in Table 2 reveal where the difficulty arises across the six question families. State identification, change detection, and composition questions are comparatively tractable, with the strongest models reaching above 85–95. In contrast, attribution and forecast questions remain difficult for all

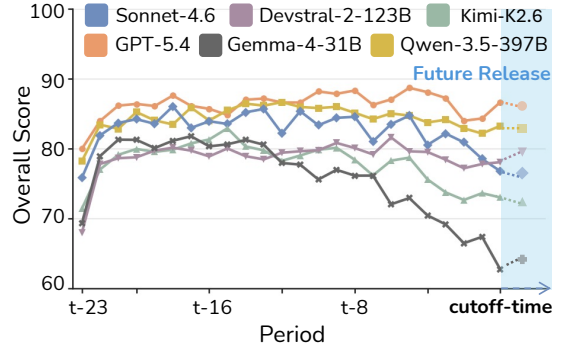


Figure 7: Live overall-score leaderboard across periods. Scores are tracked as information accumulates from $t - 23$ to cutoff time. Shaded region marks future updates.

models. The best attribution score is 46.7 and the best forecast score is 43.0. These results reveal a capability gap: *agents can compute over time series, but struggle with temporal grounding for cautious forward-looking analysis*.

4.4 Failure Mode (FM) Analysis

FM1: Reasoning failure. Figure 8 shows that reasoning failures are a major failure mode across models, involving reasoning overrun and instruction drift. Reasoning overrun occurs when a model follows a long reasoning path and times out, reflecting difficulty reaching the correct answer efficiently. Instruction drift occurs when long contexts cause

the model to forget rules, such as code-execution protocols or bash-command constraints. These failures highlight the need for efficient reasoning and memory management in evolving environments.

FM2: Temporal misuse. Temporal errors, including temporal misreading, temporal miscalculation, and inference errors stems from a weak alignment between task semantics, time series data, and evidence. Agents may read incorrect rows or time-stamps, compute incorrect variables or time windows, or rely on mismatched evidence for forecasting and attribution. These errors show that temporal grounding, rather than data access, remains a core challenge in evolving time series analysis environments.

FM3: Incomplete reporting. Incomplete reporting is a failure mode, where agents recover correct structured fields but omit required key points or fail to ground them in the report. This occurs in part because overly long reasoning chains make it difficult for agents to summarize all required points.

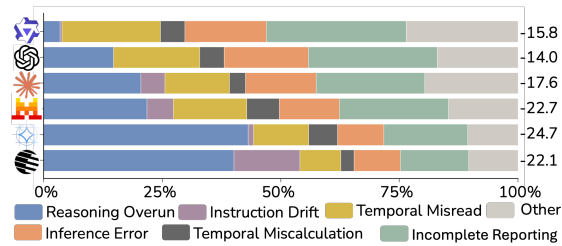


Figure 8: Failure-mode distribution per model; right numbers indicate total score impact.

4.5 The Roles of Memory and Skill Induction

Does Memory Help in Evolving Environments?

Figure 9 compares the sequential and independent evaluation of Qwen-3.5-397B and Gemma-4-31B. For Qwen-3.5-397B, memory provides a benefit: sequential evaluation outperforms independent evaluation over periods. This suggests that a persistent state helps agents reuse analysis patterns instead of solving each period from scratch. However, for Gemma-4-31B, memory is less beneficial: sequential evaluation improves early performance, but its advantage narrows and reverses in later periods. *Memory is useful only when models can manage it: history can support future analysis but may become a burden for in-context management.*

Can Agents Self-Evolve Reusable Skills? To test whether agents can convert repeated feedback into reusable analytical skills, we evaluate **TimeSage-1.0**, our smolagent-based harness

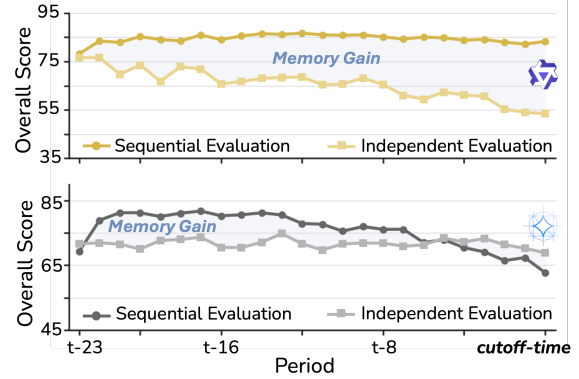


Figure 9: Sequential vs. independent evaluation for Qwen-3.5-397B (Top) and Gemma-4-31B (Bottom).

equipped with a lightweight self-evolving skill-library manager. As shown in Figure 10 a, reusable skills improve overall performance, with the clearest gains in medium- and hard-difficulty scenarios. To understand this benefit, we further examine efficiency and skill usage: enabling reusable skills reduces token cost to $0.82\times$ (Figure 10 d), while the skill-invocation frequency peaks in hard scenarios (Figure 10 b), suggesting that *reusable skills are most valuable in evolving time series analysis when reasoning is difficult*. However, skill induction and reuse are concentrated in the early periods and do not persist across the entire horizon (Figure 10 c), revealing that maintaining an effective skill library remains challenging in evolving time series tasks.

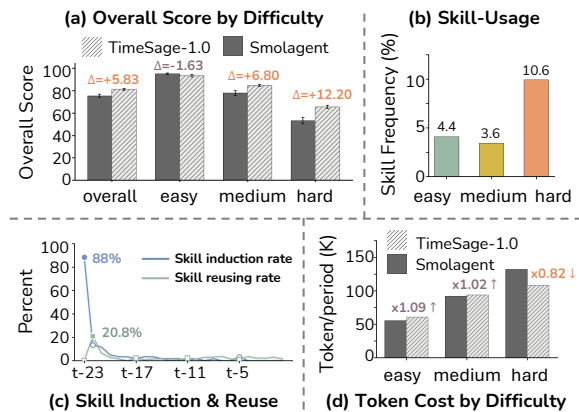


Figure 10: Analysis of self-evolving skill mechanism on Gemma-4-31B under evolving evaluation environments.

5 Conclusion

We introduced **TimeSage-EV**, a live benchmark for **agentic time series analysis in evolving environments**. TimeSage-EV formalizes recurring, cutoff-valid analytical updates and evaluates agents across 60 institutional scenarios, 1,485 scenario-

period QA pairs, and 4 complementary scoring axes. Experiments with frontier LLM agents and TimeSage-1.0 reveal persistent gaps in memory management, long-horizon reasoning, and temporal validity. These findings establish TimeSage-EV as a diagnostic benchmark for developing robust analytical agents that can update time series analyses reliably as evidence continually evolves.

Limitations

TimeSage-EV is built from public institutional sources with recurring releases, and therefore does not cover enterprise data, dashboards, or settings where historical releases are unavailable or poorly archived. Although the benchmark spans 6 domains and multiple release cadences, it cannot exhaust the diversity of document formats, institutional conventions, or analytical workflows. Furthermore, the dataset relies predominantly on English-language reports from established global institutions, which may introduce geographic and linguistic biases, and it focuses strictly on numerical time series and textual documents, excluding the visual interpretation of charts and graphs that frequently accompany real-world releases.

Regarding our evaluation protocol, we combine rule-based scoring for structured fields with LLM-based judging for report coverage, faithfulness, and quality. Despite structured rubrics and evidence grounding, some residual judge noise and bias may remain. Our experimental baseline also relies on the smolagent framework, meaning some observed failure modes, such as instruction drift or reasoning overrun, may be partially entangled with this specific harness rather than solely reflecting the underlying capabilities of the LLMs.

The experiments in this paper use May 2026 snapshot. We will release a frozen, versioned snapshot of the benchmark containing all 1,485 scenario-period instances evaluated in this paper (capped at May 2026). Future researchers can evaluate against this static version for direct comparability with our baselines, while simultaneously testing on the live branch to measure true out-of-distribution generalization on newly published institutional releases.

Intended Use

TimeSage-EV is intended as an academic research benchmark for diagnosing LLM agents on recurring time series analysis, and agent output should

be treated as decision-support signals rather than substitutes for professional judgment in high-stakes domains. Scores in TimeSage-EV should be interpreted as diagnostic indicators of analytical behavior in covered scenarios rather than as guarantees of agent reliability in unseen sources, domains, or release schedules. Agents may also produce fluent but unsupported claims or violate temporal cutoffs in ways that appear authoritative, so outputs require human verification before any downstream use.

Ethical Considerations

TimeSage-EV strictly adheres to ethical research standards. It utilizes publicly available institutional releases in compliance with each source’s usage terms. The benchmark contains only aggregate time series and reports, with no personally identifiable information (PII). Human reviewers verifying ground-truth correctness worked only with public, non-sensitive content. We emphasize that our agents’ analyses are designed as a decision-support tool for human experts, not a replacement for professional judgment in high-stakes domains, and users should be mindful of potential model hallucinations. Our approach does not involve PII or sensitive data, and we follow the Code of Ethics.

Acknowledgments

This work was financially supported by the European Union’s Horizon Europe research and innovation programme under grant agreement No. 101214398 (ELLIOT). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Commission. Neither the European Union nor the European Commission can be held responsible for them. We also acknowledge the Supercomputing Center of the Eindhoven University of Technology ([Home - TU/e Supercomputing Center](#)) for providing access to and assistance with the various computing resources available.



References

- Anthropic. 2026. Introducing claude sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>.
- Yifu Cai, Arjun Choudhry, Mononito Goswami, and Artur Dubrawski. 2024. Timeseriesexam: A time series understanding exam. *arXiv preprint arXiv:2410.14752*.
- Yifu Cai, Xinyu Li, Mononito Goswami, Michał Wilński, Gus Welter, and Artur Dubrawski. 2025. Time-seriesgym: A scalable benchmark for (time series) machine learning engineering agents. *arXiv preprint arXiv:2505.13291*.
- DeepSeek-AI. 2026. Deepseek-v4: Towards highly efficient million-token context intelligence.
- Google DeepMind. 2026. Gemma 4 model card. https://ai.google.dev/gemma/docs/core/model_card_4.
- Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. 2023. Large language models are zero-shot time series forecasters. In *Advances in Neural Information Processing Systems*, volume 36.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Live-CodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. 2024. Time-LLM: Time series forecasting by reprogramming large language models. In *International Conference on Learning Representations*.
- Ezra Karger, Houtan Bastani, Yueh-Han Chen, Zachary Jacobs, Danny Halawi, Fred Zhang, and Philip Tetlock. 2025. ForecastBench: A dynamic benchmark of AI forecasting capabilities. In *The Thirteenth International Conference on Learning Representations (ICLR)*.
- Kimi Team. 2025. Kimi K2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*.
- Yaxuan Kong, Yiyuan Yang, Yoontae Hwang, Wenjie Du, Stefan Zohren, Zhangyang Wang, Ming Jin, and Qingsong Wen. 2025. Time-MQA: Time series multi-task question answering with context enhancement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29736–29753, Vienna, Austria. Association for Computational Linguistics.
- Yaxuan Kong, Qingren Yao, Yuqi Nie, Yichen Li, Yilei Shao, Stefan Zohren, Anna Vettoruzzo, Joaquin Vanschoren, Ming Jin, and Qingsong Wen. 2026. Timesage-mt: A multi-turn benchmark for evaluating agentic time series reasoning. *arXiv preprint arXiv:2606.01498*.
- Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Mistral AI. 2025. Introducing Devstral 2 and Mistral Vibe CLI. <https://mistral.ai/news/devstral-2-vibe-cli>.
- OpenAI. 2026. GPT-5.4 thinking system card. Technical report, OpenAI.
- Qwen Team. 2026. Qwen3.5-397B-A17B. <https://huggingface.co/Qwen/Qwen3.5-397B-A17B>. Model card.
- Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. 2025. ‘smolagents’: a smol library to build great agentic systems. <https://github.com/huggingface/smolagents>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.
- Chengsen Wang, Qi Qi, Jingyu Wang, Haifeng Sun, Zirui Zhuang, Jinming Wu, Lei Zhang, and Jianxin Liao. 2025. ChatTime: A unified multimodal time series foundation model bridging numerical and textual data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 12694–12702.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*.
- Muyan Weng, Defu Cao, Wei Yang, Yashaswi Sharma, and Yan Liu. 2026. TemporalBench: A benchmark for evaluating LLM-based agents on contextual and event-informed time series tasks. *arXiv preprint arXiv:2602.13272*.
- Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, and 1 others. 2025. LiveBench: A challenging, contamination-limited LLM benchmark. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. [AutoGen: Enabling next-gen LLM applications via multi-agent conversation](#). In *Conference on Language Modeling*.
- Zhe Xie, Zeyan Li, Xiao He, Longlong Xu, Xidao Wen, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. 2025. [ChatTS: Aligning time series with LLMs via synthetic data for enhanced understanding and reasoning](#). *Proceedings of the VLDB Endowment*, 18(8).
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R. Narasimhan. 2025. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*.
- Fangxu Yu, Xingang Guo, Lingzhi Yuan, Haoqiang Kang, Hongyu Zhao, Lianhui Qin, Furong Huang, Bin Hu, and Tianyi Zhou. 2026. [TSRBench: A comprehensive multi-task multi-modal time series reasoning benchmark for generalist models](#). *arXiv preprint arXiv:2601.18744*.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and 1 others. 2024. WebArena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations (ICLR)*.

A Appendix

A.1 Task Rationale

Why recurring time series analysis is different.

TimeSage-EV targets a setting that is not captured by standard time series forecasting, static time series QA, or generic agent benchmarks. In recurring institutional analysis, the task specification remains fixed, but the evidence available to the analyst changes after each release. The agent must interpret the latest observation, compare it with historical context, use only cutoff-valid documents, and revise its report as the environment evolves.

Why temporal isolation matters. A central motivation of TimeSage-EV is to evaluate whether agents can reason within the information limit that would have been applied at a target period. Without this constraint, an agent may answer using future reports, post-hoc explanations, or documents published after the target release. TimeSage-EV prevents this by exposing cumulative time series data and prior-period source documents while withholding the target-period release for ground truth construction and evidence verification.

Why agentic evaluation matters. Real analytical workflows require more than reading a fixed table. Agents must inspect data files, use tools, consult documents, preserve state across periods, and produce both structured answers and natural-language reports. TimeSage-EV evaluates agentic time series analysis as an integrated workflow, exposing failures such as unsupported causal claims, poor use of evidence, and incomplete reporting.

A.2 Core Concepts

Live benchmark. We call TimeSage-EV a live benchmark because its evaluation set continuously expands as institutional releases are published. For each scenario s , new releases extend $\mathcal{T}_s$ over time. Each extension updates the cumulative time series and the available prior-period evidence, withholds the target-period release, and keeps the task contract fixed. This expansion is automated rather than manual: each scenario carries a self-contained, re-runnable pipeline that re-pulls the source, rebuilds the cumulative inputs, withholds the newest release, and re-derives ground truth and evidence under the scenario’s frozen rules. A monthly maintenance script executes this pipeline across all scenarios, keeping TimeSage-EV live without re-authoring any questions and ground-truths.

Evolving environment. An evolving environment is a sequence of period-specific evidence for the recurring analytical task. The domain background, question template, and answer schema remain fixed within a scenario, while the time series extends, and the document set changes as the releases arrive. The agent must update its judgments for periods while respecting the cutoff point.

Agentic time series analysis. Agentic time series analysis refers to workflows in which an agent uses tools to inspect time series data, consult external evidence, answer analytical questions, and produce task-specific output. TimeSage-EV studies a distinctive setting in which the analytical task recurs across an evolving environment: the time series extends, the document set changes, and each period imposes a cutoff on evidence. The agent can maintain evidence, reasoning traces, or reusable analysis artifacts over periods. We therefore evaluate not only the structured answer, but also a free-form report that exposes the agent’s evidence use, temporal grounding, and reasoning. This mirrors real-world time series analysis, where conclusions are typically delivered as periodic analytical reports rather than standalone structured predictions.

A.3 Benchmark Construction

Section 3.2 and Figure 4 summarize the benchmark generation pipeline. This appendix expands that overview into a detailed construction protocol. Sections A.3.1 through A.3.4 walk through the pipeline sequentially, from data collection and scenario generation to quality control and human annotation. Section A.3.5 documents the data sources and reports volatility statistics. Finally, Section A.3.6 presents three illustrative scenario examples.

A.3.1 Stage 1: Data Collection

Step 1: Source registry. The pipeline is a domain-by-domain pass loop. Each domain pass begins from a target domain selected to close a current portfolio gap, such as an under-represented tier or release cadence. For that domain, the builder constructs a registry of candidate public sources, searching for recurring institutional data products that could fill the gap. We prioritize government agencies, central banks, international organizations and stable institutional publishers, including sources such as EIA, NOAA, CDC, FRED, BLS, the World Bank and IEA. Each candidate is recorded in eight standardized metadata fields:

endpoint, authentication requirement, update frequency, historical span, data format, license, report links, and free-form notes. The step yields a per-domain candidate registry and exits when the domain has accumulated at least ten plausible candidates or is exhausted, with every metadata field either filled in or explicitly marked unknown.

Step 2: Attribute filtering. Taking the candidate registry as input, we first reject sources that cannot support a recurring benchmark task before writing any concrete question, and then assign a difficulty tier to the passed sources.

- Metadata filtering: apply Table 3 to remove sources that are inaccessible, weakly time-indexed, insufficiently variable, or unsuitable for evaluation of evolving-agents.
- Tier criteria: assign each surviving source to preliminary tiers based on Table 4: *Easy* for categorical state identification, *Medium* for temporal-pattern summarization, and *Hard* for report-grounded reasoning about causes or outlooks.

Table 3: Metadata-level validity checks in Step 2.

No.	Validity check
1	Public access. Source is publicly accessible without credentials, payment, or approval.
2	Time series grounding. Task uses an observable time-stamped series with values for the queried period.
3	Query persistence. The same template applies to both historical and future periods.
4	Temporal anchoring. The source provides a date, period, timestamp, or update window.
5	GT freshness. Ground truth updates at least monthly, except report-level hard tasks.
6	Temporal variability. Outputs vary over time and avoid near-constant majority labels.
7	Evolving-agent fit. Task requires time-dependent retrieval, update, comparison, or synthesis.

Step 3: Execution validation. The third step takes each retained source–tier pair and verifies by execution so that it can produce valid period-level benchmark instances. The builder retrieves the primary time series, releases archive, normalizes it into a period-indexed table, and retrieves the corresponding source releases, such as dashboards, briefs, bulletins, or analytical reports. As summarized in Table 5, each candidate must provide at least 24 periods aligned instances with traceable, time-isolated, and rule-evaluable evidence.

The difficulty tier assigned in Step 2 is confirmed during execution: *Easy* scenarios must yield cate-

Table 4: Difficulty-tier criteria across source, task, answer format, and evaluation in Step 2.

Tier	Criteria
Easy	Source: a data dashboard reporting current operational state as categorical indicators. Task: identify the current state (reasoning LOW, capability dependence LOW). Format: a single categorical label. Eval: exact match against the correct option.
Medium	Source: a data brief with time series and descriptive commentary, but no causal analysis. Task: summarize recent changes across time points and sub-components (reasoning MEDIUM, dependence MEDIUM). Format: a structured schema (period value, direction, leading sub-component, short summary). Eval: LLM judge or keyword matching for per-field coverage and factual consistency.
Hard	Source: an analytical report with trend judgments, drivers, risks, and outlooks. Task: reason about causes of change and what comes next (reasoning HIGH, dependence HIGH). Format: a structured schema (conclusion, key drivers, risks, uncertainties, evidence). Eval: LLM judge or keyword matching, with each claim traceable to a section, table, or figure.

gorical state-identification instances, *Medium* scenarios must yield summaries of temporal patterns, and *Hard* scenarios must yield report-grounded reasoning about causes or outlooks. Sources that pass the metadata checks in Step 2 but fail any execution-level check are rejected and recorded in the rejection log with the failure reason.

A.3.2 Stage 2: Scenario Generation

Step 4: Evidence collection. Taking each instance skeleton validated in Step 3 as input, the fourth step assembles the evidence state that an agent will see at evaluation time, which also serves as the traceable substrate for deriving ground truth. We preserve the raw source downloads and extract an evidence state comprising two parts: the cumulative time series analysis reports and the cumulative time series table. For period t_k , the report set contains all reports from the earliest available period through t_{k-1} , while the time series table contains all rows from the earliest available period through t_k . The output is therefore a time-isolated evaluation state: the agent receives the cumulative table and the prior-period source documents, while the target-period report is withheld for ground-truth construction. Each scenario yields at least 24 such per-period states, one per instance.

The evidence is accepted only if it passes two

Table 5: Execution-level validity checks in Step 3.

No.	Validity check
1	Period alignment. The time series value, source release, and question instance refer to the same timestamp, date, period, or update window.
2	Minimum history. The source supports at least 24 periods for backtesting and future live evaluation.
3	Reference traceability. Time series analyses in released reports are traceable to concrete public evidence, such as a field, label, value, table, figure, or report statement.
4	Information isolation. The evidence package for period t uses only information available at or before the query time, excluding future releases, later revisions, and retrospective explanations.
5	Objective evaluability. Time series analyses in released reports can be derived by predefined rules and evaluated without relying on subjective preference.

checks: (1) **Completeness**, ensuring sufficient coverage, temporal continuity, and plausible numeric scale; and (2) **Column hygiene**, keeping only genuine raw measurements while removing duplicate, constant, textual, rescaled, or already-derived fields unless explicitly allowed.

Step 5: Question generation. Using the evidence from Step 4 and the assigned difficulty tier, Step 5 defines a scenario-level evaluation contract: a fixed question template, answer schema, and scoring rule shared across all periods. Concretely, we use an LLM (Claude Opus 4.6) to match each question intent in Table 6 against the time series analysis reports and extract the corresponding analysis paragraphs, which the next step transforms into the answer schema and scoring rule listed in Table 7.

In the meantime, the question coverage in difficulty tier is enforced: Easy uses at least one A-group tag; Medium uses at least three tags across A/B/C/D plus a universal summary; and Hard uses at least three tags across E/F/D plus a summary and traceable evidence. Before admission, each contract must pass three checks: (1) recurrence across sampled and full periods, (2) synchronization among tags, schema fields, and scoring rules, and (3) artifact grounding against the cumulative table, instance folder, and answer schema. Contracts that fail any check are rejected.

Step 6: Ground-truth extraction. Step 6 produces a reproducible, leak-free canonical answer. For tasks whose answers can be derived directly from the time series, such as state identification or change detection, we use an LLM (Claude Opus

Table 6: Question taxonomy in step 5: 27 fine-grained question types in six main categories (A–F).

ID	Question intent
<i>A — State identification</i>	
A1	The current categorical state of the target.
A2	The latest reported value.
A3	The direction of the latest reading (up / down / flat).
<i>B — Change detection</i>	
B1	The change from the previous period.
B2	The direction of the period-over-period change.
B3	The magnitude of the recent multi-period trend.
B4	How many consecutive periods the trend has lasted.
B5	The deviation from a declared baseline (e.g., five-year average).
B6	Whether this period is a record extremum within a time window.
B7	The ordinal rank of this period within its history.
<i>C — Composition identification</i>	
C1	The set of leading contributors (top- N).
C2	The single dominant contributor.
C3	The breakdown of the whole into components with their values or shares.
C4	The assignment of entities into named buckets.
<i>D — Event detection</i>	
D1	The anomalies or unusual values flagged this period.
D2	The classification or enumeration of discrete events.
<i>E — Attribution</i>	
E1	The single cause attributed to the change.
E2	The primary driver among several.
E3	The secondary drivers.
E4	The set of drivers, each paired with traceable evidence.
E5	The stakeholders affected by the change.
E6	The issuer’s forward guidance or committed response.
E7	A comparison with a named earlier period or season.
<i>F — Outlook</i>	
F1	The forward risks, paired with traceable evidence.
F2	The direction, regime, or tone of the outlook.
F3	The outlook with its confidence and horizon.
F4	A conditional outlook (“if X, then Y”).

Table 7: Scoring-rule assignment by question type in step 5. Rule details are defined in Table 11.

Scoring rule	Question types
<i>Atomic rules</i>	
Label match	A1, A3, B2, B6, C2, D2, E1, E2, E5, F2
Numeric check	A2, B1, B3, B4, B5, B7
Set overlap	C1, C4, D1, D2, E3, E5
LLM judgment	E1, period summary
<i>Composition rules</i>	
Joint scoring	E6, E7, F3, F4
Item-wise scoring	C3, E4, F1

4.6) to generate executable code (e.g., a rule-based state identification script or a change-rate calculation) that yields verifiable answers. For tasks whose answers must be inferred jointly from textual context and the time series, we also use the LLM to transform the evidence extracted in Step 5 into the structured schema. This procedure constructs the ground truth from the given questions under the fixed contract, yielding a verifiable per-period task.

The target-period release is used only for ground-truth construction and verification, never as agent-visible evidence. Thus, each field is either rule-reproducible from the cumulative table or explicitly grounded in the withheld source. Finally, the prompt is rendered from the same scenario contract, specifying the task, inputs, rules, requested fields, and JSON schema.

A.3.3 Stage 3: Quality Control

Step 7: Grounding audit. Step 7 verifies that the question, ground-truth, evidence, data, and source documents are mutually consistent and solvable under the intended information boundary. The audit applies eight gates covering schema, rules, data, evidence, and cross-instance consistency (Table 8); a scenario passes only when all gates have zero-failed outcomes.

Table 8: The eight grounding-audit gates in Step 7.

Gate Grounding-audit check	
1	Schema conformance. Ground truth validates against the answer schema; its top-level keys equal the required set, and enum values stay within the declared options.
2	File completeness. Each instance carries its question, ground truth, evidence, time series, and schema files.
3	Question-artifact consistency. Columns, files, and period labels named in the question exist and match the data; output-example keys and enum options equal the schema.
4	Rule reproducibility. The deterministic derivation rule recomputes the stored ground truth (report-grounded fields exempt).
5	Time series health. The cumulative table is strictly ascending in time.
6	Evidence-data consistency. Numeric and key-value references in the evidence resolve to table cells, and ground-truth enum strings appear in the evidence.
7	Quote grounding. Every verbatim quote in the supporting evidence is a substring of the cited source document.
8	Cross-instance health. Instance count matches the scenario, ground-truth fields vary across periods, and each table’s last row matches its folder period.

Step 8: Human check. The final step adjudicates audit warnings and catches design violations that are difficult to formalize. Human re-

viewers decide whether each warning is by design, re-check artifact quality against the full gate of Table 9, and remove leakage references. By-design warnings may be released, such as hard-tier evidence warnings, whitelisted constants, or benign period-formatting differences, whereas scenarios are blocked or quarantined if they contain target-period leakage, revision-framing leakage, column-hygiene defects, or instance-count defects. After the human check, accepted scenarios are promoted into the benchmark, portfolio statistics are recomputed, and the termination condition $\text{total} \geq 60 \wedge \text{domains} \geq 5 \wedge \min(\text{tier}) \geq 10$ is re-checked; if it is not met, the loop returns to Step 1 for the next domain pass.

Table 9: The nine-section artifact quality gate in Step 8.

No.	Artifact quality check
1	Time series completeness. Core columns are populated, the date axis is continuous, and there are at least 24 rows (cf. Step 4).
2	Column hygiene. No duplicate date parts, rescaled duplicates, constant columns, embedded narrative, or agent-derivable columns.
3	GT leakage. No column’s latest value, value set, or sign restates the ground-truth answer (Easy categorical state exempted).
4	Difficulty-task alignment. Easy reads a latest-row state; Medium summarises across time from a brief; Hard reasons causally from a report, with no analytical conclusions baked into the table.
5	Question non-hallucination. Referenced fields and windows exist; the question recurs across at least three periods and instantiates over all periods; difficulty matches the tier.
6	Schema-template consistency. Tier and domain match the directory, Easy options equal the schema enum, and declared columns equal the table header.
7	Directory compliance. Required scenario, environment, script, source, and per-instance files are present, with at least 24 instances.
8	GT traceability. The latest period’s evidence cites concrete source locations, and no field is inferable from the table alone (cf. Step 6).
9	Information isolation. No table rows or evidence citations postdate the instance period (V7).

A.3.4 Human Annotations

We recruit three human reviewers to evaluate the quality of TimeSage-EV. All reviewers are PhD students in Computer Science with at least full professional proficiency in English, and all consent to the use of their annotations for benchmark construction and analysis. The reviewers examine scenario specifications, period-specific questions, structured ground truth, and supporting evidence. The review focuses on whether each question is clear and

answerable under the cutoff, whether the ground-truth fields are schema-conformant and correct, and whether each answer is grounded in the cumulative time series or cutoff-valid source documents. Disagreements are resolved through discussion, and a scenario is released only after both automatic checks and human review pass. The following are the detailed instructions to guide the reviewers:

Thank you for helping verify the quality of TimeSage-EV, a benchmark for recurring time series analysis under a temporal cutoff. Please follow the instructions below.

For each scenario you will receive: a scenario specification, period-specific questions, an answer schema, the ground truth (ground_truth.json) with its evidence (gt_evidence.md), the cumulative timeseries.csv, and any prior-period source documents.

Instructions

1. Review without help from other people, answer-generation tools, or search engines beyond the provided documents. There is no time limit.
2. For each scenario, check:
 - (a) Clarity: Is the question clear, recurring, and answerable for the target period?
 - (b) Cutoff validity: Can the answer be obtained without the target-period source or any future evidence?
 - (c) Schema conformance: Do all ground-truth fields match the schema and allowed values?
 - (d) Correctness: Are the structured answers correct for the target period?
 - (e) Grounding: Is every field supported by the time series or cutoff-valid documents?
 - (f) Variation: Does the scenario vary across periods enough for recurring evaluation?
3. If a scenario is unclear, incorrect, or ambiguous, flag it with the affected period, field, and a short explanation.
4. Return one of: PASS, PASS_WITH_MINOR_FIXES, or FAIL.

Thank you for your contributions!

A.3.5 Released Sources

After the construction loop, Table 10 lists the sources that pass metadata filtering, execution validation, ground-truth extraction, automatic audits, and human review. These sources form the released TimeSage-EV corpus, which contains 60 scenarios across six domains, including agriculture, climate, energy, finance, healthcare, and transportation, and three tiers, with 20 scenarios per tier.

Table 10: Public data sources for all 60 TimeSage scenarios, grouped by domain. Tier: E = easy, M = middle, H = hard. Scenario names are shortened to the shortest unique prefix.

T	Scenario	Cad.	Source
Agriculture			
E	usda_cattle	mon.	esmis.nal.usda.gov
E	usda_chicken	mon.	esmis.nal.usda.gov
E	usda_cold_storage_butter	mon.	esmis.nal.usda.gov
E	usda_cold_storage_pork	mon.	esmis.nal.usda.gov
M	usda_crop	wkly.	esmis.nal.usda.gov
M	usda_drought	wkly.	droughtmonitor.unl.edu
M	usda_wasde	mon.	usda.gov
H	fao_food	mon.	fao.org
H	usda_fas	mon.	apps.fas.usda.gov
H	usda_us	mon.	ams.usda.gov; fred.stlouisfed.org
Climate			
E	noaa_cpc_ao	mon.	cpc.ncep.noaa.gov
E	noaa_cpc_mjo	daily	psl.noaa.gov
E	noaa_cpc_oni	mon.	cpc.ncep.noaa.gov
E	noaa_cpc_pna	mon.	cpc.ncep.noaa.gov
M	c3s_copernicus	mon.	climate.copernicus.eu
M	noaa_ncei_global	mon.	ncei.noaa.gov
M	noaa_ncei_us	mon.	ncei.noaa.gov
H	iri_enso	mon.	iri.columbia.edu
H	noaa_cpcenso	mon.	cpc.ncep.noaa.gov
H	noaa_cpc_monthly	mon.	cpc.ncep.noaa.gov
Energy			
E	eia_refinery	wkly.	eia.gov
E	eia_us_commercial	wkly.	eia.gov
E	eia_wti	daily	fred.stlouisfed.org
M	eia_natural_gas_monthly	mon.	eia.gov
M	eia_us_petroleum	mon.	eia.gov
M	eia_wpsr	wkly.	eia.gov
H	eia_monthly	mon.	eia.gov
H	eia_natural_gas_weekly	wkly.	eia.gov
H	iea_oil	mon.	iea.org
H	ief_comparative	mon.	ief.org
Finance			
E	cboe_vix	daily	fred.stlouisfed.org
E	fred_sahm	mon.	fred.stlouisfed.org
E	fred_us	daily	fred.stlouisfed.org
M	dallas_fed	mon.	dallasfed.org
M	fred_consumer	mon.	fred.stlouisfed.org
M	philly_fed	mon.	philadelphiafed.org
H	bea_pce	mon.	fred.stlouisfed.org
H	boe_monetary	8/yr	bankofengland.co.uk
H	ecb_monetary	8/yr	ecb.europa.eu
H	frb_us	mon.	fred.stlouisfed.org
Healthcare			
E	cdc_nwss	wkly.	data.cdc.gov
E	cdc_us	wkly.	data.cdc.gov
E	hhs_us	wkly.	data.cdc.gov
M	bls_jolts	mon.	fred.stlouisfed.org
M	bls_us	mon.	fred.stlouisfed.org
M	ecdc_cdtr	wkly.	ecdc.europa.eu
M	paho_measles	biwk.	paho.org
H	bls_medical	mon.	fred.stlouisfed.org
H	cdc_resp	wkly.	data.cdc.gov
H	who_cholera	mon.	who.int
Transportation			
E	bls_diesel	wkly.	fred.stlouisfed.org
E	bts_airline	mon.	fred.stlouisfed.org
E	tsa_checkpoint	daily	tsa.gov
M	ata_truck	mon.	fred.stlouisfed.org
M	bls_ppi	mon.	fred.stlouisfed.org
M	eurocontrol_monthly	mon.	eurocontrol.int
M	portla_monthly	mon.	portoflosangeles.org
H	cass_freight	mon.	cassinfo.com
H	freightos_weekly	wkly.	freightos.com
H	iata_air	mon.	iata.org

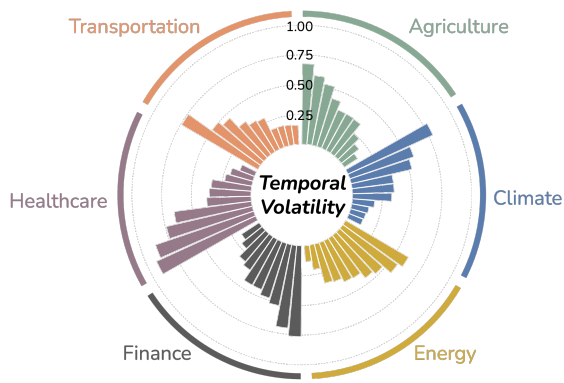


Figure 11: Per-Scenario Answer Volatility Across Domains.

We quantify how much each environment evolves over time using the per-period answer transition rate (Figure 11), which measures how often a scenario's structured answer fields change between consecutive periods. Across all 60 scenarios the rate spans 0.13 to 0.91, confirming that temporal volatility is not uniform but a scenario-level property the benchmark deliberately varies.

A.3.6 Scenario Examples

To illustrate the range of tasks covered by our benchmark, Figures 12–14 present three representative scenarios spanning the Easy, Medium, and Hard difficulty tiers. Each scenario casts the model as a domain analyst who must read a cumulative time-series snapshot and analysis reports and answer a structured set of questions about the latest reporting period.

The Easy scenario (Weekly US Diesel Price-Band Classification) in Figure 12 asks the model to read the most recent row of an EIA/FRED diesel price series and assign it to one of four fixed price bands, followed by a brief summary. This is a task that relies on data lookup and threshold logic.

The Medium scenario (Monthly Healthcare Labor Market Analysis) in Figure 13 draws on BLS JOLTS data for the Health Care and Social Assistance sector and additionally requires the model to compute month-over-month and year-over-year changes, infer their direction, and synthesize a short digest, thereby introducing multi-step quantitative reasoning over the time series.

The Hard scenario (Monthly ENSO Climate Analysis), shown in Figure 14, is the most open-ended. Working from NOAA CPC's monthly

Weekly US Diesel Price-Band Classification

Easy Transportation Weekly Cutoff: 2026-05

Task

You are a **freight-and-trucking-cost analyst** tracking the weekly national-average US #2 diesel pump price published by the U.S. Energy Information Administration (EIA) and mirrored on FRED as series 'GASDESW' — the all-sellers national average retail diesel price in USD per gallon. **Each week you read the latest Monday observation off the FRED snapshot and report which fixed price band it falls into.**

Input

'/root/timeseries.csv' is the cumulative weekly history of the GASDESW series sampled at the Monday observation date (the EIA publishes the survey result every Monday for the prior Monday's nationally averaged sample) Columns:

- 'observation_date' — the Monday date (YYYY-MM-DD) on which EIA reported the national-average pump price.
- 'iso_week' — ISO week label of that Monday, e.g. '2026-W20'.
- 'price_usd_per_gal' — US national-average retail #2 diesel pump price, USD per gallon.

Questions

The diesel price band is a fixed-threshold classification on the latest row's 'price_usd_per_gal':

- 'price_usd_per_gal < 3.50' -> 'cheap'
- '3.50 <= price_usd_per_gal < 4.25' -> 'normal'
- '4.25 <= price_usd_per_gal < 5.00' -> 'elevated'
- 'price_usd_per_gal >= 5.00' -> 'extreme'

[1] Question: Read the latest row of '/root/timeseries.csv' and report **which retail diesel price band US #2 diesel pump prices fall into this week.**
 Answer field: 'diesel_price_band' (string)
 Options: 'cheap' / 'normal' / 'elevated' / 'extreme'

[2] Question: **Wrap up with a 1–3 sentence factual summary** covering the current period, the 'diesel_price_band' tag, and at least one supporting numeric value from the latest CSV row.
 Answer field: 'report'

Figure 12: Easy scenario: Weekly US Diesel Price-Band Classification.

Monthly Healthcare Labor Market Analysis

Middle

Healthcare

Monthly

Cutoff: 2026-05

Task

You are acting as a **U.S. healthcare-labor-market analyst** writing a short descriptive digest of BLS's monthly ***Job Openings and Labor Turnover Survey*** (JOLTS) release for the Health Care and Social Assistance super-sector over a stretch of consecutive data months. **Each turn you describe Health-Care job openings, the month-over-month change and direction, and the year-over-year percent change.**

Input

`"/root/timeseries.csv"` is the cumulative BLS JOLTS Health Care & Social Assistance (NAICS 62) history from the earliest in-window month through the current data month, ascending in time, one row per data month. Columns:

- `'as_of_data_month'` — data month, 'YYYY-MM'.
- `'job_openings_thousands'` — float, JOLTS health-care + social assistance job openings (FRED JTS6200JOL), thousands of persons.
- `'hires_thousands'` — float, JOLTS hires (JTS6200HIL), thousands.
- `'quits_thousands'` — float, JOLTS quits (JTS6200QUL), thousands.
- `'total_separations_thousands'` — float, JOLTS total separations (JTS6200TSL), thousands.

`"/root/gt_source/"` is a directory of prior-release BLS JOLTS news-release PDFs, named `'<YYYY-MM>.pdf'` (one PDF per data month).

Questions

[1] Question: **What is the JOLTS Health-Care & Social Assistance job-openings level for this data month ?**
Answer field: `'job_openings_thousands'` (number)

[2] Question: **What is the month-over-month change in job openings, and what direction does that imply?**
Answer field: `'mom_change_thousands'` ([`'number'`, `'null'`])
Answer field: `'mom_direction'` ([`'string'`, `'null'`])
Options: `'expanding'` / `'contracting'` / `'stable'` / `'None'`

[3] Question: **What is the year-over-year % change in job openings versus the same data month a year ago?**
Answer field: `'yoy_change_pct'` ([`'number'`, `'null'`])

[4] Question: **Wrap up with a 2-3 sentence summary covering openings level, MoM change/direction, and the YoY % move.**
Answer field: `'report'`

Figure 13: Medium scenario: Monthly Healthcare Labor Market Analysis.

Monthly ENSO Climate Analysis

Hard

Climate

Monthly

Cutoff: 2026-04

Task

You are acting as an **operational climate analyst** preparing a structured synthesis of NOAA CPC's monthly ****ENSO Diagnostic Discussion**** (DD), co-issued with IRI on the 2nd Thursday of each month, across a stretch of consecutive monthly issues. **Each month you take the latest released DD and report CPC's ENSO alert-system status, the probabilistic 3-month outlook, the dominant ocean atmosphere drivers, the uncertainties CPC flags, a short factual summary.**

Input

`"/root/timeseries.csv"` is the cumulative ONI history from DJF 1950 through the data month of the current DD issue, ascending in time, one row per overlapping 3-month season. Columns:

- `'season'` — 3-letter overlapping 3-month code (DJF, JFM, FMA, ...).
- `'year'` — year of the season's centered month.
- `'centered_month'` — ISO month of the season center.
- `'nino34_total_c'` — Niño 3.4 total SST (°C).
- `'nino34_anomaly_c'` — the ONI value (Niño 3.4 SST anomaly, °C, vs the 1991-2020 base period).

`"/root/gt_source/"` is a directory of ***prior months*** CPC ENSO Diagnostic Discussion HTML files, named `'<YYYY-MM>.shtml'`.

Questions

[1] Question: **What is the current ENSO alert-system status this month?**
Answer field: `'enso_alert_status'` (string)
Options: `'El Niño Advisory'` / `'La Niña Advisory'` / `'El Niño Watch'` / `'La Niña Watch'` / `'Final El Niño Advisory'` / `'Final La Niña Advisory'` / `'Not Active'`

[2] Question: **What is CPC's 3-month outlook — phase, probability, and horizon?**
Answer field: `'outlook_3mo'` (object)

[3] Question: **Which 3+ ocean atmosphere drivers best explain this month's call?**
Answer field: `'key_drivers'` (array, minItems 3, objects)

[4] Question: **List at least 3 uncertainties relevant to this month's ENSO outlook.**
Answer field: `'risks_uncertainties'` (array, minItems 3, objects)

[5] Question: **Which event/anomaly tokens characterize this period?** List 0-3 items from the closed vocabulary.
Answer field: `'event_anomaly_callouts'` (array, maxItems 3, unique strings)
Options: `'el_nino_advisory'` / `'la_nina_advisory'` / `'enso_watch'` / `'spring_barrier'` / `'subsurface_anomaly'` / `'trade_wind_anomaly'` / `'convection_shift'` / `'model_consensus'`

[6] Question: **Wrap up with a 1–3 sentence summary covering the alert status, outlook, and 1–2 drivers or risks.**
Answer field: `'report'`

[7] Question: **List at least 3 pieces of supporting evidence backing your analytical claims.**
Answer field: `'supporting_evidence'` (array, minItems 3, objects)

Figure 14: Hard scenario: Monthly ENSO Climate Analysis.

ENSO Diagnostic Discussion Reports alongside the Niño 3.4 ONI time series history, the model must report six things: the current alert-system status, a three-month outlook, the dominant ocean–atmosphere drivers, any flagged uncertainties, event tokens drawn from a closed vocabulary, and a grounded summary with supporting evidence.

A.4 Evaluation Methodology

We evaluate each agent run along four independent axes to disentangle answer correctness from report quality: structured-answer accuracy on the one hand, and report coverage, faithfulness, and writing quality on the other. This section first defines the evaluation unit in Section A.4.1, then detail the evaluation method for each axis in Section A.4.2.

A.4.1 Evaluation Unit

The atomic unit of evaluation is a (*scenario, period*) instance. For each instance, the agent produces two artifacts: a structured answer that fills the scenario answer schema and a free-form report that summarizes the answer. The answer schema decomposes each period time series analysis into a set of keypoints corresponding to the elementary facts that a competent analysis must establish. The report serves as an analytical summary, integrating these keypoints and their evidence into human-readable text. We evaluate the two artifacts along four axes, each scored on a 0–100 scale: keypoint accuracy (ACC), keypoint coverage (COV), report faithfulness (FAITH), and report quality (QUAL). Averaging them gives the overall instance score:

$$\text{AGG} = \text{mean}(\text{ACC}, \text{COV}, \text{FAITH}, \text{QUAL}).$$

A.4.2 The Four Evaluation Axes

Keypoint accuracy. This axis measures how well the structured answer matches the ground truth (GT). Each keypoint is scored by comparing the agent-provided value with the GT value according to the answer-type-specific rules in Table 11. The match score reflects the degree of agreement between the two values, and the final score is the average across keypoints, rescaled to 0–100.

Keypoint coverage. This axis measures the report’s completeness: whether it communicates the content of each required keypoint. For each keypoint, the judge assigns one of three labels: YES, PARTIAL or NO, which are scored as 1.0, 0.5 and 0.0, respectively. The axis score is obtained by

Table 11: Primitive match rules. Each rule compares the agent output a with the ground-truth value b , or output set A with ground-truth set B .

Rule	Match score
Label match	1 if $a = b$, else 0.
Boolean match	1 if $\text{bool}(a) = \text{bool}(b)$, else 0.
Text match	1 if stripped strings are equal, else 0.
Numeric check	1 if $ a - b \leq \tau_{\text{abs}}$, or $ a - b / b \leq \tau_{\text{rel}}$ when relative tolerance is set and $b \neq 0$; else 0. With no tolerance, exact equality is required.
Set overlap	Dice/F1 overlap $2 A \cap B /(A + B)$; two empty sets score 1, and a single empty set scores 0.

averaging these keypoint-level scores within the instance and rescaling the result to the range 0–100. During the evaluation, the easy and medium tiers contain no reasoning questions. Therefore, a keypoint anchored to a ground-truth scalar is marked covered without an LLM call. In contrast, the hard-tier uses the LLM-based evaluation path, since a verbatim scalar match alone does not guarantee that the underlying concept has been conveyed. The coverage judge prompt is given in Appendix A.5.2.

Report faithfulness. This axis measures factual groundedness: the fraction of the report’s atomic claims supported by the sources. The judge first decomposes the report into a small set of atomic single-fact claims and then verifies each against a layered reference: the scenario keypoints (level 1), excerpts of scenario context (level 2), and general world knowledge (level 3). A claim grounded at any level counts as supported; claims that contradict a keypoint or are mere speculation count as unsupported. The axis is the fraction of supported claims, on a scale of 0–100. The judge prompts are presented in the Appendix A.5.2.

Report quality. This axis measures writing quality alone, namely genre conformance, structure, and readability. The judge scores the report against the scenario tier’s rubric, which lists the genre, the signals a good report should contain, those it should avoid, and an exemplar; it marks each required signal as fully hit, partially hit, or missed, and flags any forbidden signal. The axis rewards hits, half-credits partial hits, deducts violations, and normalizes by the number of required signals, on a scale 0 – -100. Factual errors are not penalized here since the other axes handle them. The quality judge prompt is given in the Appendix A.5.2.

A.4.3 Evaluation Examples

Figure 15 shows a successful run on the BTS Airline Load Factor scenario. The agent correctly fills the structured answer (is_year_high_load_factor = false), matching the ground truth and producing a perfect keypoint precision. Its report communicates every required keypoint—the 80.5% load factor for 2024-11 and the negative high-load flag—so coverage is also full. In faithfulness, each atomic claim is grounded either in the keypoints (TS) or the scenario context (DOC), with no unsupported speculation. The report additionally hits all required quality signals while avoiding forbidden ones, such as causal attribution or fabricated press-release citations. All four axes score 100, giving an aggregate of AGG = 100.

Figure 16 shows a failure on the WASDE Crop Balance scenario, where the agent over-commits to a single commodity. It answers corn for the dominant U.S. commodity (GT: mixed) and reports a concrete ending stock level where the ground truth is N / A, so several precision keypoints score 0; only kp_us_mom_pct matches. Coverage is correspondingly partial: the report conveys the level and month-over-month keypoints, but omits the event/anomaly callouts and the correct dominant-commodity framing. Faithfulness remains high, as the stated claims are individually grounded in the keypoints or context. Quality is satisfied but only half-credited on the signal requiring both the latest level and the signed monthly change. The axis scores (20, 60, 100, 87.5) average to AGG = 66.88, illustrating how the four axes jointly separate answer correctness from report quality.

A.5 Prompt Templates

This appendix reports the prompt templates used by TimeSage-EV. Scenario-specific prompts are generated from these templates and the scenario specification. The fully rendered period prompts for all 1,485 period-level instances are released with the benchmark files.

A.5.1 Agent Prompt

System and sandbox instruction. At each period, the agent receives a task prompt together with the following sandbox instruction. scenario specific fields such as required answer keys, tier, and optional web-search availability are filled out from the scenario specification.

BTS Airline Load Factor

Questions

It's now the 2024-11 BTS airline passenger load factor release. I need a brief on this release:

- Is this month's U.S. system-wide passenger load factor the highest reading in the trailing 12 months?
- Wrap up with a 1–3 sentence

Agent Output

"is_year_high_load_factor": false,
"report": "For 2024-11, the U.S. system-wide passenger load factor was 80.5% in timeseries.csv.
is_year_high_load_factor is False because the latest value is not strictly greater than each of the previous 11 months."

Evaluation

Keypoint (Accuracy)	output	gt	Score
is_year_high_load_factor	false	false	100

Keypoint (Coverage)	Match	Score
For 2024-11, the U.S. system-wide passenger load factor was 80.5% in timeseries.csv.	✓	100
is_year_high_load_factor is False	✓	100

Atomic Fact (Faithfulness)	Verdict	Score
U.S. system-wide passenger load factor for 2024-11 was 80.5%.	Doc	100
is_year_high_load_factor for 2024-11 is False.	TS	100
The data source is timeseries.csv.	Doc	100
The load factor value for 2024-11 is not strictly greater than each of the previous 11 months	TS	100

Key Signals (Quality)	Match	Score
U.S. system-wide passenger load factor for 2024-11 was 80.5%.	✓	100
is_year_high_load_factor for 2024-11 is False.	✓	100

Judge Rationale

correctly names the month as 2024-11 and states is_year_high_load_factor as False, with no forbidden speculation, causal attribution, or BTS press release citation.

(100 + 100 + 100 + 100) / 4 = 100

Figure 15: Successful case of agent output.

WASDE Crop Balance

Questions

It's now the April 2024 WASDE release. I need a brief on this month's U.S. + world ending-stocks picture:

- Which U.S. commodity saw the largest absolute revision to ending stocks this month?
- What was its latest U.S. ending-stocks level?
- What was its signed month-over-month % change?
- Is the same commodity's world ending-stocks picture tightening, loosening, or stable this release?
- Which event/anomaly tokens characterize this period?
- Wrap up with a 2-4 sentence summary covering the dominant U.S. commodity and one notable cross-commodity comparison.

Agent Output

```
"dominant_us_commodity": "corn",
"dominant_us_latest_mbu": 2127.0,
"kp_us_mom_pct": 0.00
"dominant_world_direction": "loosening",
"event_anomaly_callouts": [],
"report": "In the March 2026 WASDE, corn had the largest U.S. ending-stocks revision at 2127 million bushels, +0.00% from the prior release. For the same commodity, world ending stocks were 292.75 MMT in March versus 288.98 MMT in 2026-02, indicating loosening. Across commodities, March U.S. ending-stocks revisions were corn +0.00%, soybeans +0.00%, and wheat +0.00%."
```

Evaluation

Keypoint (Accuracy)	output	gt	Score
dominant_us_commodity	corn	mixed	0
dominant_us_latest_mbu	2127	N/A	0
kp_us_mom_pct	0	0	100
dominant_world_direction	loosening	stable	0
event_anomaly_callouts	[]	[supply_increase, supply_decrease, stocks_build]	0

Keypoint (Coverage)	Match	Score
dominant_us_commodity	✗	0
2127 million bushels	✓	100
revisions were corn +0.00%	✓	100
indicating loosening	✓	100
event_anomaly_callouts	✗	0

Atomic Fact (Faithfulness)	Verdict	Score
The report is from the March 2026.	Doc	100
The corn U.S. ending-stocks revision was +0.00% from the prior release.	TS	100
World ending stocks for corn were 292.75 MMT in March..	Doc	100
The March 2026 U.S. ending-stocks revision for ... was +0.00%.	TS	100

Key Signals (Quality)	Match	Score
Names the WASDE release month explicitly	✓	100
States dominant_us_commodity using controlled enum.	✓	100
Includes dominant_us_latest_mbu (mbu) & signed dominant_us_mom_pct	🟡	50
States dominant_world_direction using controlled enum	✓	100

(20 + 60 + 100 + 87.5) / 4 = 66.88

Figure 16: Failed case of agent output.

You are a markets/data analyst. All files you need are in the CURRENT WORKING DIRECTORY.

The cwd contains:

- * timeseries.csv -- the historical data series
- * answer_schema.json -- required JSON shape for your answer
- * gt_source/<prior>.* -- prior-period reports (PDF/HTML/JSON)

Available tools:

- * Python code execution
- * final_answer(answer=<dict>, report=<str>) -- submit your final output. You must call this tool exactly once to finish. Do not write the answer to a file. Do not emit a final textual reply instead of calling the tool.
- * web_search(query: str, n: int=5) -- audited, time-isolated, enabled only for scenarios with audited cutoff-valid retrieval.

Sandbox rules:

Authorized imports: pandas, numpy, json, csv, re, math, statistics, datetime, pathlib, io, collections, itertools, functools, pypdf.

Forbidden builtins: open, eval, exec, compile, __import__, globals, locals.

Forbidden modules include os, sys, subprocess, socket, urllib, requests.

File / IO replacements:

- * Read text: Path('x.txt').read_text()
- * Read JSON: json.loads(Path('x.json').read_text())
- * Read CSV: pd.read_csv('x.csv')
- * Read PDF: PdfReader('x.pdf').pages[0].extract_text()
- * List dir: [p.name for p in Path.cwd().iterdir()]

Scenario tier: {tier}

Required answer keys: {required_keys}

Task rules:

1. Read the CSV and prior-period gt_source files if needed. Do not hallucinate values.
2. Never cite or read the current period's gt_source. Only prior-period source documents are allowed.
3. Your final answer dict must contain exactly the required keys. Your report must cite specific CSV cells or prior-period files.
4. End your run by calling final_answer(answer=..., report=...).

Memory instruction. For sequential evaluation, the following memory instruction is appended.

Memory management is optional in multi-period episodes. If your conversation

memory becomes bloated with failed code attempts, long observations, or patterns already learned, you may call `request_memory_compaction(reason=...)`. The framework will summarize what you have learned across past periods after the current period finishes, then reset memory to that summary before the next period. Do not call this for a single-period task.

Skill prompt. When the skill library is enabled, a skill-index block is prepended to each period's task prompt. The block has two forms, depending on whether the library is empty or populated. An agent may save a reusable function with `induct_skill` and inspect the full body of a saved skill with `retrieve_skill`; saved skills become callable as live tools starting the next period.

When the library is empty, the following seeding nudge is shown.

No skills have been saved yet for this sweep. You are the first agent to run.

If you write a small, reusable idiom this period (a schema-specific CSV loader, a date-to-period converter, a band classifier, a row lookup), save it with `induct_skill` so later periods can call it as a tool.

Good candidates: small single-purpose functions returning a clean Python value.
Bad candidates: scenario-specific one-off math, or the literal answer dict.

Induct the skill BEFORE calling `final_answer`, while the code is fresh.

When there is at least one skill, the reuse-enforcement form is shown instead.

```
=== Available skills ===
The following skills are already registered
as tools for this period.
```

INVOKE THESE SKILLS -- do NOT re-implement a listed skill inline. Reuse is faster, less error-prone, and consistent across periods.

Example call:
 `result = {first_skill_name}(...)`

Skills:
 - {skill_name} -- {skill_description}
 ...

Use `retrieve_skill(name=...)` to read a skill's notes and full code before

invoking if its description is not clear enough. If you find a new reusable idiom this period, save it with `induct_skill`.

Period question prompt. Each period-level question is rendered from the scenario's question template.

It is now {period_label}. I need an evidence-based analysis for this period's {release_name}.

Use `/root/timeseries.csv` and any prior-period files in `/root/gt_source/`. The current-period source document is not available.

Please answer the following questions:

1. {field_question_1}
2. {field_question_2}
- ...
- N. {field_question_N}

Return a JSON object conforming to `/root/answer_schema.json` and a concise natural-language report grounded in cutoff-valid evidence.

A.5.2 Evaluation Prompts

Keypoint coverage judge. For report coverage, deterministic literal matching is used when possible. Otherwise, the LLM judge receives the following prompt for each keypoint.

You are checking whether a report communicates a predefined content unit (ACU). Reply with exactly one of: YES, PARTIAL, NO.

Content unit:
 description: {keypoint_description}
 verification question:
 {verification_question}

Report:
 {agent_report}

Reply with the single word YES, PARTIAL, or NO and nothing else.

Report faithfulness decomposition. For report faithfulness, the report is first decomposed into atomic claims.

Decompose this report into AT MOST 10 atomic, verifiable claims. Each claim must be ONE scalar fact: one number, one named entity, one date, one direction, one classification -- never a compound sentence

with multiple facts.

Skip markdown headers, bullet labels, rhetorical sentences, hedges, and definitional preambles. Each claim should be standalone and grounded enough to verify against a data source.

Return a JSON list of strings, no prose around it.

Report:
{agent_report}

Report faithfulness verification. Each atomic claim is then verified against cutoff-valid reference sources.

Verify a single atomic factual claim against reference sources.

Claim:
{atomic_claim}

Reference sources:
{reference_sources}

Pick exactly one verdict from this set:
tier1_supported (keypoint says so),
tier2_supported (context says so),
tier3_general_knowledge (only if the claim is encyclopedic and not contradicting any reference),
contradicts_keypoint (clearly conflicts with a keypoint),
speculation (unsupported by any reference).

Return JSON: {"verdict": "<one>"}

Report quality judge. Report quality is scored separately from factual correctness.

You are scoring a report's WRITING quality only -- its genre conformance, structure, and readability. DO NOT score for factual correctness; that is evaluated separately by a different scorer.

Tier genre:
{tier_genre}

Required signals:
- {required_signal_1}
- {required_signal_2}
...

Forbidden signals:
- {forbidden_signal_1}
- {forbidden_signal_2}
...

Exemplar of a good report:
{quality_exemplar}

Report to score:
{agent_report}

Return JSON:
{
 "required_hits": ["<signal>", ...],
 "required_partial": ["<signal>", ...],
 "required_misses": ["<signal>", ...],
 "forbidden_violations": ["<signal>", ...],
 "rationale": "<one sentence>"
}

A.6 Agent Harnesses

A.6.1 Baseline Harness: smolagent

Agent framework. All agents are built on smolagents, a lightweight framework in which an LLM drives a multi-step loop of tool calls, each step appended to a running memory of prior actions and observations. We use its code-agent variant, where the model acts by writing executable Python rather than emitting structured tool-call arguments, which suits the data-manipulation nature of time series analysis and exposes a single uniform action channel across all models.

Action surface. The baseline agent is a smolagent Code Agent in which Python is the action language: the agent emits code, the sandbox executes it, and the resulting observation becomes the next turn. Imports are restricted to a whitelist of analytical libraries, file access is mediated through path-based readers rather than raw file handles, and all networking modules are excluded by design, so the only sanctioned external access is the audited web-search tool. No default tools are added beyond those the protocol requires: the agent can submit its output, request memory compaction, and issue web searches.

Memory management. Memory management uses a hybrid policy: a compaction is scheduled at the next period boundary whenever the agent requests that one or the most recent step's input-token count exceed a fixed threshold. When triggered, the harness issues a single call to the agent's own model to summarize the CSV layout, working code idioms, schema pitfalls, feedback patterns, and a one-line recap per past period, then re-seeds the conversation with that summary.

Internet access. Our harness blocks all network libraries (e.g., urllib, requests, socket). To preserve a broad action space, we expose a web-search tool as the sole network channel, which

returns only historical archived snapshots through a three-stage pipeline: (1) Candidate discovery: SearXNG³ resolves the query into a set of candidate URLs; (2) Archive retrieval: each URL is queried against Common Crawl⁴ with server-side temporal filtering, so that only captures predating the cutoff are returned; (3) Fallback: if no qualifying capture exists, the Wayback Machine⁵ supplies the snapshot closest to the cutoff, which is discarded if dated after it. These stages guarantee that every retrieved page was archived strictly before the cutoff date.

A.6.2 Self-Evolving Harness: TimeSage-1.0

Why Self-Evolve. TimeSage-EV evaluates agents on a recurring analytical task whose evidence changes over time, raising a question that single-period evaluation cannot answer: *can an agent improve future analyses by converting feedback and repeated reasoning patterns from earlier periods into reusable procedures?* A standard smolagent harness with tool use and memory carries previous context, but mixes reusable routines with transient observations, failed code attempts, and period-specific facts, so memory alone cannot distinguish a generally useful workflow from merely accumulated history.

A diagnostic harness. TimeSage-1.0 addresses this question by keeping the agent interface, model configuration, task exposure, time budget, and evaluation protocol aligned with the baseline, while adding only a lightweight self-evolving skill-library manager. This isolates the effect of reusable skills from that of changing the base model or supplying additional evidence. It is therefore a diagnostic condition rather than an optimized production agent, measuring whether agents can *induce*, *validate*, *retrieve*, and *reuse* compact analysis routines across the repeated release periods of a scenario.

Skill-library tools. TimeSage-1.0 augments the baseline smolagent harness with a skill-library layer, leaving the sandbox, inputs, and required artifacts unchanged. It adds two tools: `induct_skill` saves a small reusable Python function with a name, description, and notes when the agent identifies a recurring analysis idiom, and `retrieve_skill` returns the full body of a saved skill when its short description is insufficient for reliable use.

³<https://searxng.org>

⁴<https://commoncrawl.org>

⁵<https://web.archive.org>

Chronological registration. Accepted skills become callable tools from the next period, so a skill induced in the period t_k cannot alter the current answer but can affect behavior in t_{k+1} and beyond. This preserves the chronological structure of the evolving environment.

Prompting and reuse. The library is exposed through a period-level skill-index block. When empty, it seeds the agent toward compact reusable idioms, such as schema-specific loaders or row lookups, and away from one-off arithmetic and literal answer dictionaries. Once non-empty, the block lists the available skills, registers them as tools, and instructs the agent to invoke an existing skill rather than re-implement it inline. The prompt templates are given in the Appendix A.5.

Lightweight quality control. Before a skill is exposed in later periods, a check enforces that it is a callable, self-contained routine with a usable interface and a faithful description, filtering out candidates that are overly period-specific, duplicate an existing skill, or encode a literal answer. The check is deliberately minimal, keeping the library usable during evaluation without turning skill construction into a separate supervised process.

A.7 Additional Results

A.7.1 Breakdown of Main Results

Table 12 provides a detailed breakdown of the performance of the model by difficulty tier, complementing the aggregate results reported in the main text. For each model, we report scores along four evaluation dimensions (Accuracy, Coverage, Faithfulness, and Quality) as well as the Overall score, separately for the Easy, Medium, and Hard tiers. Cell shading is computed within each tier-and-metric group, so colors are comparable across models within a column but not across tiers; the best entry in each group is shown in bold.

Two patterns are worth highlighting. First, performance degrades consistently from Easy to Hard across all models and dimensions: while most models exceed 90 Overall on the Easy tier, scores drop substantially on the Hard tier (from 72.70 for GPT-5.4 down to 53.02 for Gemma-4-31B), indicating that the harder questions remain a discriminating factor. Second, the dimension-level breakdown reveals that this degradation is driven primarily by Accuracy and Coverage, whereas Faithfulness and Quality remain comparatively stable. This suggests

Table 12: Per-difficulty-tier scores across evaluation dimensions. Cell shading uses a shared scale within each tier-and-metric group (low    high); the best result in each group is shown in bold.

Model	Tier	Accuracy	Coverage	Faithfulness	Quality	Overall
 GPT-5.4	Overall	84.89	77.30	91.20	90.58	85.99
	Easy	98.54	98.67	96.12	93.04	96.59
	Medium	93.44	76.79	96.64	87.28	88.54
	Hard	62.09	56.58	80.44	91.69	72.70
 Qwen-3.5-397B	Overall	84.72	75.57	88.66	87.99	84.24
	Easy	98.61	99.86	92.50	97.61	97.14
	Medium	92.48	78.52	91.86	82.79	86.41
	Hard	62.53	48.21	81.38	84.01	69.03
 Sonnet-4.6	Overall	82.76	74.32	84.46	87.97	82.38
	Easy	99.47	97.99	93.95	97.19	97.15
	Medium	89.03	75.91	87.21	82.43	83.65
	Hard	59.36	49.03	72.04	84.75	66.29
 Kimi-K2.6	Overall	77.88	70.07	81.74	81.90	77.90
	Easy	98.75	98.65	94.59	97.97	97.49
	Medium	82.89	69.88	81.22	74.40	77.10
	Hard	51.68	41.82	69.52	74.00	59.25
 Devstral-2-123B	Overall	74.14	69.08	84.43	81.37	77.26
	Easy	89.70	92.28	93.82	93.30	92.27
	Medium	73.11	66.81	79.34	67.03	71.57
	Hard	59.76	48.44	80.56	84.96	68.43
 Gemma-4-31B	Overall	75.05	69.08	80.87	76.25	75.31
	Easy	95.03	99.42	93.82	91.37	94.91
	Medium	82.23	70.76	83.79	74.48	77.81
	Hard	47.43	37.07	64.82	63.09	53.10

that on difficult inputs models tend to produce well-formed and faithful outputs that nonetheless miss or misidentify the correct content. GPT-5.4 and Qwen-3.5-397B are the strongest overall, while the gap between models widens on the Hard tier.

A.7.2 LLM Judge Verification

To validate the reliability of our LLM judge, we conducted a human–judge agreement study on 300 instances (5 sampled periods per scenario) of GPT-5.4’s agent outputs scored by DeepSeek-V4-Flash. Each instance was independently labeled by a human annotator—given the question, reference, and prediction—using the same discrete label space as the LLM judge: coverage (YES/PARTIAL/NO per keypoint), faithfulness (five fixed verdicts per atomic claim), and quality (hit/partial/miss/violation per signal). As shown in Table 13, coverage

Table 13: Human–LLM judge agreement across the three evaluation axes.

Evaluation Axis	Agreement Rate	Cohen’s κ
Coverage	94.5%	0.882
Faithfulness	98.4%	0.917
Quality	76.0%	0.665

and faithfulness exhibit strong alignment with human judgments, while quality agreement is lower yet still substantial. It is likely because quality assessment is inherently more subjective, whereas the other two axes follow clearer, more objective rubrics. These results indicate that the LLM-judge setup can serve as a reliable proxy for human evaluation under our rubric.

A.8 AI Usage

We used AI assistance during the writing and editing process to improve the clarity, organization, and wording of the manuscript. The authors controlled scientific content, benchmark design, experimental analysis, and final writing decisions. AI tools were not used to generate benchmark ground truth without automatic and human verification. All reported results, figures, and tables are based on the authors' benchmark artifacts and evaluation pipeline.